

ProcessPilot AI - Complete Engineering Journey

1. Opening: Why I Started Building This

I started building ProcessPilot AI because I wanted to explore whether I could bring organizational knowledge, conversations, documents, and operational context together and make that information queryable through an AI interface.

I noticed a common problem: documents contain useful knowledge, meetings produce action items, and tasks exist separately from knowledge. I wanted to see if I could build a centralized system that bridges these isolated silos into one relational, conversational brain.

2. Phase 1: Defining the Problem

What I initially identified

- Organizational information is distributed across isolated PDFs.
- Meetings produce action items that get lost immediately after the call ends.
- Conventional keyword search cannot understand semantic relationships.
- Organizational relationships (who wrote what, who manages whom) are difficult to query.

What I decided the system needed

Fragmented documents -> Centralized ingestion + vector retrieval (RAG)

Meeting information gets lost -> Transcript processing + task extraction

Organizational relationships are difficult to query -> Graph-based organizational context

3. Phase 2: Writing the PRD

Before implementing the system, I defined what the product was supposed to solve. I wanted an AI copilot capable of retrieving documents, summarizing meetings, and tracking extracted tasks for Admins, Managers, and Employees.

Initially Planned: A basic chatbot over PDFs.

Actually Implemented: A multi-agent orchestration system that handles compliance checking, graph querying, and task generation via WebSockets.

4. Phase 3: From PRD to TRD

I translated these product requirements into technical requirements. I chose React 19 + Vite for the frontend to ensure fast client-side rendering. For the backend, I needed asynchronous capabilities critical for LLM streaming, so I chose FastAPI (Python). For the database, I required both relational data (RBAC) and vector storage, leading me to PostgreSQL (Supabase) with the pgvector extension.

5. Phase 4: System Architecture Before Implementation

Initial Architecture

I initially planned to use an in-memory NetworkX graph stored as a JSON file, and local HuggingFace embeddings.

Limitation Discovered

Stateful in-memory JSON graphs do not work in stateless, horizontally scaled serverless environments (like Render). Local embeddings consumed too much memory on the free tier.

Architectural Change

I migrated the graph to SQL-backed tables (kg_nodes, kg_edges). I replaced local embeddings with a deterministic hash-based mock generator for dev, and Gemini for production.

6. Phase 5: Building the Foundation

The actual development sequence was:

1. Project structure & FastAPI scaffolding.
2. PostgreSQL Database & SQLAlchemy Models.
3. Authentication & RBAC (Admin, Manager, Employee).
4. Document Ingestion pipeline.
5. RAG (Embeddings & pgvector).
6. Multi-Agent routing layer (CEOAgent).
7. Graph migration to SQL.
8. Meeting intelligence & task extraction.
9. Security (PII Redaction, API Key AES Encryption).
10. Deployment (Render & Vercel).

7. Phase 6: Database Design

I started from the reasoning: What information does this system need to persist? Users need to belong to Departments. Documents need to be chunked into Embeddings. Meetings need to generate Tasks. The most significant evolution was adding the kg_nodes and kg_edges tables to replace the legacy NetworkX implementation.

8. Phase 7: Building Document Ingestion

The pipeline: Raw File -> PyMuPDF Extraction -> PII Redaction Regex -> Semantic Chunking -> Embedding Generation -> PGVector Storage.

Problem Encountered: PII leakage. I realized I could not just send raw HR PDFs to OpenAI. I built pii_redactor.py using regex to scrub SSNs and Credit Cards before vectorizing.

9. Phase 8: Building RAG

The flow: Question -> Retrieval -> Context -> LLM -> Answer.

I started with simple Cosine Similarity. I then realized keyword search was still important for specific names/IDs, so I added BM25Okapi to create a Hybrid Retrieval system.

Planned improvement (Not Yet Implemented): Reciprocal Rank Fusion (RRF) to better combine the BM25 and Vector scores.

10. Phase 9: Building the Multi-Agent System

A single AI pipeline was not enough. If someone asked Who wrote this?, vector search failed. I evolved the architecture to: User question -> CEOAgent (Routing Layer) -> Specialized Sub-Agents (SearchAgent, GraphAgent, SOPAgent) -> LLM.

Note: This is specialized agent orchestration (routing). Fully autonomous, unconstrained tool-calling loops are NOT currently implemented.

11. Phase 10: Knowledge Graph

Vector search answers What is semantically similar? but misses relational questions like Which department owns this?.

Initial approach: NetworkX in-memory JSON.

Limitation: Fails on ephemeral serverless disks.

Migration: Moved to SQL-backed kg_nodes.

12. Phase 11: Meeting Intelligence

Flow: Transcript -> LLM Analysis -> Structured JSON -> Action Items -> Tasks.

Problem: LLMs hallucinate conversational filler.

Solution: I implemented strict Regex extraction to isolate the JSON payload before parsing it into database Tasks. Added WebSockets to push tasks live to users.

13. Phase 12: Security Was Built Alongside

Authentication -> RBAC -> ABAC (Department isolation) -> PII protection (Regex) -> API-key encryption (AES-GCM).

Limitation: WebSocket authentication currently relies on query parameters, which is weaker than HTTP bearer headers. Formal SOC2 is not certified.

14. Phase 13: Real Engineering Problems

Problem: Groq Model Deprecations

What happened: Groq frequently decommissioned fast models, causing 404 crashes.

Solution: Built a dynamic fallback iterator in llm_client.py that catches 404s, queries Groq for active models, and automatically retries.

Problem: Postgres Timezone Crashes

What happened: Postgres returns timezone-aware datetimes. Sorting logs in analytics.py using naive Python datetimes caused HTTP 500 errors.

Solution: Built a sorting algorithm that strictly normalizes all datetimes to UTC.

15. Phase 14: Testing & Observability

I realized AI systems fail differently from CRUD apps. I implemented pytest for ABAC, auth, and ingestion. For observability, I created ai_failures, llm_usage, and agent_logs tables to track API timeouts, token costs, and routing decisions.

16. Phase 15: Deployment

Local Dev -> FastAPI on Render -> Supabase PostgreSQL.

Problem: Render Free Tier has an ephemeral disk. File uploads vanish.

Solution: Uploads are immediately processed into chunks/vectors in PostgreSQL.

17. Phase 16: What I Have Completed Today

I have successfully deployed a system featuring Hybrid RAG (PGVector + BM25), a Multi-Agent Delegation Architecture, a SQL-Backed Knowledge Graph, Real-time Task generation via WebSockets, and PII Redaction.

18. Phase 17: What I Did Not Complete

- NOT IMPLEMENTED: Reciprocal Rank Fusion (RRF).
- NOT IMPLEMENTED: True autonomous agent tool-calling loops.
- NOT VERIFIED: Formal SOC2/GDPR compliance.
- LEGACY: chromadb is configured but superseded by pgvector.

19. Phase 18: What I Would Build Next

- Next Iteration: Implement RRF for better hybrid search scoring.
- Medium Term: S3 bucket storage for durable PDF archiving.
- Long Term: Deep RAG evaluation tracking (like RAGAS) and true autonomous loops.

20. Final Section: What This Project Taught Me

Building this taught me that architectural prototypes often do not survive deployment constraints (e.g., migrating from NetworkX to SQL). I learned that AI systems require deep observability because they fail silently or hallucinate. Finally, I learned that security (PII redaction) must be placed at the very start of the ingestion pipeline, not as an afterthought.

21. Final Status Matrix

Vector Store: PostgreSQL pgvector (IMPLEMENTED)

Knowledge Graph: SQL-Backed kg_nodes (IMPLEMENTED)

RAG Retrieval: Hybrid Vector + BM25 (IMPLEMENTED)

RRF: Planned (NOT IMPLEMENTED)

Real-time Updates: WebSockets (IMPLEMENTED)

Security: ABAC + PII + AES Crypto (IMPLEMENTED)

Autonomous Agents: Delegation Routing (PARTIALLY IMPLEMENTED)